

LAPORAN TUGAS 2

Sistem Rekomendasi Produk

Graph Traversal & Pattern Matching dengan Neo4j



Dosen Pengampu: Dr. Hendra, S.Si., M.Kom.

Nama Anggota:

1. Akmal	H071241065
2. Muhammad Hairi	H071241055
3. Shabrina Zahrah Ramadhani	H071241045
4. Trivania Buli Karoma	H071241039

**DEPARTEMEN MATEMATIKA
PROGRAM STUDI SISTEM INFORMASI
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS HASANUDDIN
2026**

BAB 1 — PENDAHULUAN

1.1 Tujuan

Tugas ini bertujuan agar mahasiswa mampu melakukan Graph Traversal (penelusuran graph) untuk menemukan pola tersembunyi, yang merupakan kekuatan utama dari Neo4j dalam membangun sistem rekomendasi.

1.2 Studi Kasus

Sebuah platform e-commerce ingin merekomendasikan produk kepada pengguna berdasarkan pola pembelian pengguna lain yang serupa. Metode yang digunakan adalah Collaborative Filtering, yaitu menemukan user dengan kesamaan pembelian lalu merekomendasikan produk yang belum dibeli oleh user target.

1.3 Tools yang Digunakan

- Neo4j Community Edition 2026.05.0
- Java JDK 21.0.11
- Neo4j Browser (localhost:7474)
- Cypher Query Language (CQL)

BAB 2 — DESAIN GRAPH

2.1 Label Node

Graph terdiri dari 2 label node:

Label	Property	Contoh Nilai	Keterangan
User	name (String)	Alice, Bob, Charlie	Merepresentasikan pengguna e-commerce
Product	name (String)	Laptop, Mouse, Keyboard, Monitor	Merepresentasikan produk yang dijual

2.2 Tipe Relasi

Relasi yang digunakan adalah PURCHASED dengan arah dari User ke Product:

Relasi	Arah	Keterangan
PURCHASED	(User) → (Product)	Menunjukkan bahwa User telah membeli Product

2.3 Data Dummy

User	Produk yang Dibeli
Alice	Laptop, Mouse
Bob	Laptop, Mouse, Keyboard
Charlie	Monitor

BAB 3 — IMPLEMENTASI

3.1 Inisialisasi Database

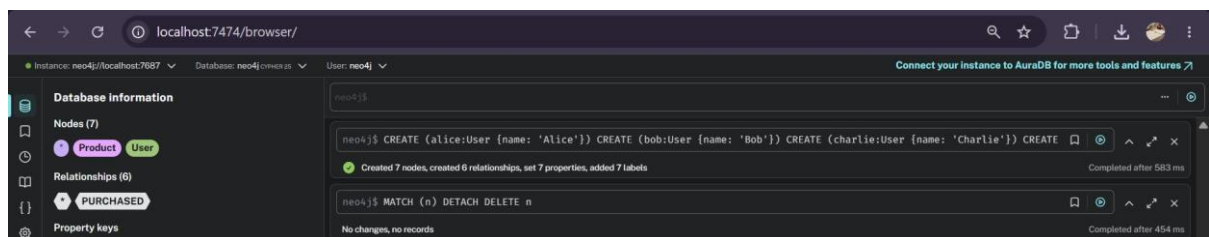
Langkah pertama adalah membersihkan database dan membuat semua node beserta relasinya menggunakan query Cypher berikut:

```
// Bersihkan database
MATCH (n) DETACH DELETE n;

// Buat Node User & Product + Relasi PURCHASED
CREATE (alice:User {name: 'Alice'}),
(bob:User {name: 'Bob'}),
(charlie:User {name: 'Charlie'}),
(laptop:Product {name: 'Laptop'}),
(mouse:Product {name: 'Mouse'}),
(keyboard:Product {name: 'Keyboard'}),
(monitor:Product {name: 'Monitor'}),
(alice)-[:PURCHASED]->(laptop),
(alice)-[:PURCHASED]->(mouse),
(bob)-[:PURCHASED]->(laptop),
(bob)-[:PURCHASED]->(mouse),
(bob)-[:PURCHASED]->(keyboard),
(charlie)-[:PURCHASED]->(monitor);
```

Hasil eksekusi: Created 7 nodes, created 6 relationships, set 7 properties, added 7 labels.

Screenshot hasil inisialisasi data:

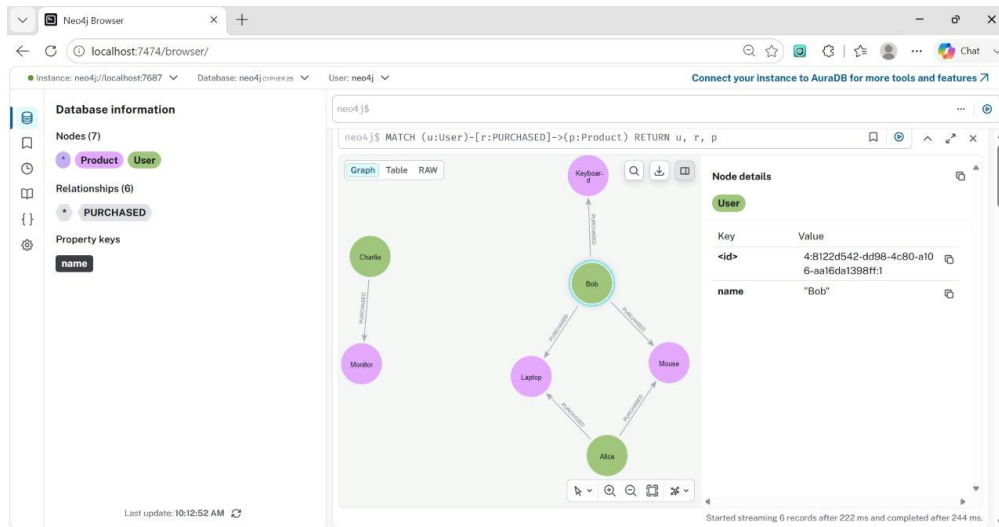


3.2 Visualisasi Graph

Setelah data dibuat, visualisasi graph dapat dilihat dengan query:

```
MATCH (u:User)-[:PURCHASED]->(p:Product) RETURN u, r, p
```

Screenshot visualisasi graph:



3.3 Query Mencari Similar User

Langkah pertama adalah mencari user lain yang memiliki kesamaan pembelian dengan Alice:

```
MATCH (alice:User {name: 'Alice'})-[:PURCHASED]->(p:Product)
<-[:PURCHASED]-(similarUser:User)
WHERE alice <> similarUser
RETURN DISTINCT similarUser.name AS similar_user;
```

Screenshot hasil query similar user:



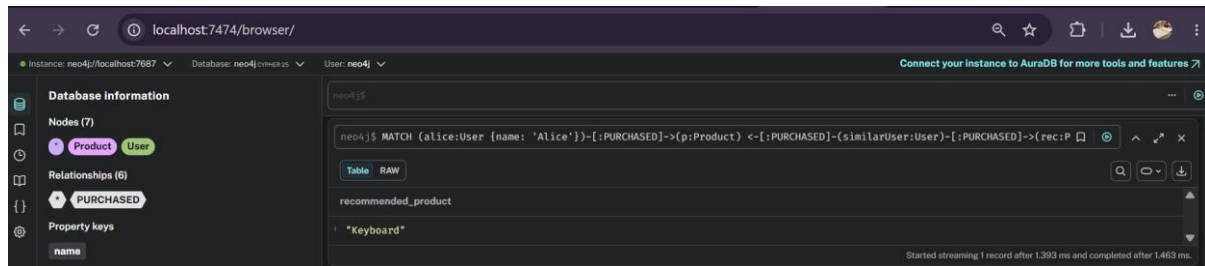
Hasil: Bob — karena Bob juga membeli Laptop dan Mouse seperti Alice.

3.4 Query Rekomendasi Utama

Query berikut mengimplementasikan Collaborative Filtering untuk merekomendasikan produk kepada Alice:

```
MATCH (alice:User {name: 'Alice'})-[:PURCHASED]->(p:Product)
<-[:PURCHASED]-(similarUser:User)-[:PURCHASED]->(rec:Product)
WHERE alice <> similarUser
AND NOT (alice)-[:PURCHASED]->(rec)
RETURN DISTINCT rec.name AS recommended_product;
```

Screenshot hasil query rekomendasi utama:



3.5 Penjelasan Logika Query

Query di atas bekerja dengan 3 tahap traversal graph:

- Tahap 1: Temukan semua produk (p) yang pernah dibeli oleh Alice
- Tahap 2: Dari produk yang sama, temukan user lain (similarUser) yang juga membelinya
- Tahap 3: Dari similarUser, temukan produk (rec) yang belum pernah dibeli Alice

Pattern traversal: (alice)→(p)←(similarUser)→(rec)

3.6 Query Bonus — Ranking Rekomendasi

Query bonus mengurutkan rekomendasi berdasarkan jumlah user lain yang juga membeli produk tersebut (degree/bobot relasi):

```
MATCH (alice:User {name: 'Alice'})-[:PURCHASED]->(p:Product)
      <-[:PURCHASED]-(similarUser:User)-[:PURCHASED]->(rec:Product)
WHERE alice <> similarUser
      AND NOT (alice)-[:PURCHASED]->(rec)
RETURN rec.name AS recommended_product,
       COUNT(DISTINCT similarUser) AS score
ORDER BY score DESC;
```

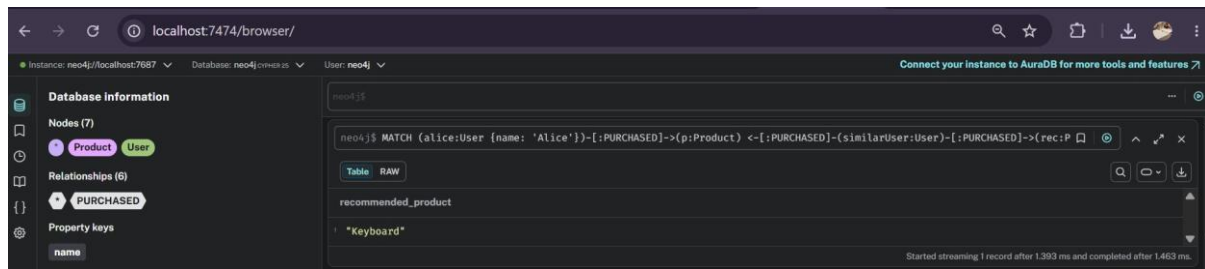
BAB 4 — HASIL DAN ANALISIS

4.1 Hasil Query Rekomendasi

Hasil eksekusi query rekomendasi untuk user Alice:

recommended_product
Keyboard

Screenshot hasil query rekomendasi:

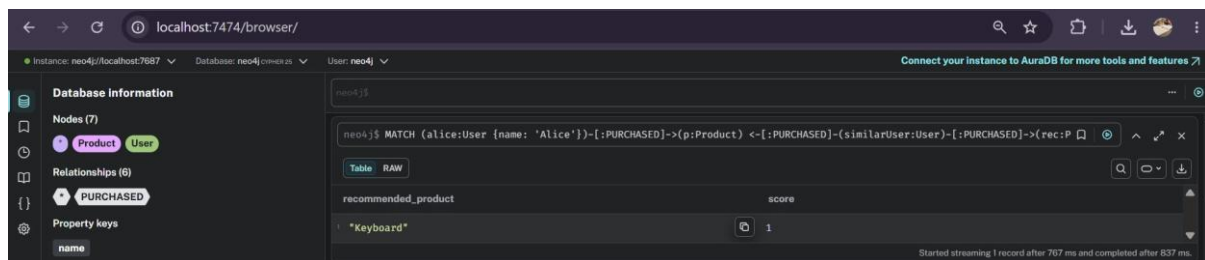


Query berhasil merekomendasikan produk Keyboard untuk Alice.

4.2 Hasil Query Bonus (dengan Ranking)

recommended_product	score
Keyboard	1

Screenshot hasil query bonus:



4.3 Analisis Hasil

Dari hasil query dapat dianalisis sebagai berikut:

- Alice membeli: Laptop dan Mouse
- Bob membeli: Laptop, Mouse, dan Keyboard — memiliki kesamaan dengan Alice (Laptop & Mouse)
- Bob adalah similar user dari Alice karena membeli produk yang sama
- Keyboard dibeli Bob tetapi belum dibeli Alice, sehingga direkomendasikan
- Score = 1 karena hanya ada 1 similar user (Bob) yang membeli Keyboard
- Charlie tidak menjadi similar user karena tidak ada produk yang dibeli bersama Alice

BAB 5 — KESIMPULAN

Tugas ini berhasil mengimplementasikan sistem rekomendasi produk berbasis Graph Traversal menggunakan Neo4j dengan metode Collaborative Filtering. Beberapa poin kesimpulan:

- Graph database Neo4j sangat efektif untuk menemukan pola hubungan antar data melalui traversal relasi

- Cypher Query Language memungkinkan pencarian pola (pattern matching) yang kompleks dengan sintaks yang intuitif
- Metode Collaborative Filtering berhasil mengidentifikasi similar user (Bob) dan merekomendasikan produk yang relevan (Keyboard) untuk Alice
- Query bonus dengan COUNT dan ORDER BY berhasil menghitung degree/bobot relasi untuk meranking rekomendasi

LINK GITHUB

Source code dan seluruh file tugas dapat diakses pada repository berikut:

<https://github.com/doraemonn24/neo4j-recommendation>